I'm working on a terminal Connect Four game (Python, no external libraries) with
these files: main.py, game.py, board.py, ai.py. Here is the code:

--- board.py ---

```
ROWS, COLS = 6, 7


class Board:
    def __init__(self):
        self.grid = [["."] * COLS for _ in range(ROWS)]

    def drop(self, col, token):
        if not 0 <= col < COLS:
            return None
        for r in range(ROWS - 1, -1, -1):
            if self.grid[r][col] == ".":
                self.grid[r][col] = token
                return r
        return None

    def full(self):
        return all(self.grid[0][c] != "." for c in range(COLS))

    def winner(self, token):

        directions = [(0, 1), (1, 0)]
        for r in range(ROWS):
            for c in range(COLS):
                if self.grid[r][c] != token:
                    continue
                for dr, dc in directions:
                    if all(
                        0 <= r + i * dr < ROWS and
                        0 <= c + i * dc < COLS and
```

```
                    self.grid[r + i * dr][c + i * dc] == token
                    for i in range(4)
                ):
                    return True
        return False

    def print(self):
        print("\n  " + " ".join(str(i + 1) for i in range(COLS)))
        for row in self.grid:
            print("  " + " ".join(row))

--- game.py ---
from board import Board
from ai import AI


class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four — you are X.")
        while True:
            self.board.print()
            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()
                if raw == "q":
                    return
                try:
                    col = int(raw) - 1
                except ValueError:
                    print("Enter a column number.")
                    continue
            else:
                col = self.ai.choose_column(self.board)

            if col is None or self.board.drop(col, self.turn) is
None:
```

```
            print("Column unavailable.")
            if self.turn == "O":
                return
            continue

        if self.board.winner(self.turn):
            self.board.print()
            print(self.turn, "wins!")
            return
        if self.board.full():
            self.board.print()
            print("Draw.")
            return

        self.turn = "O" if self.turn == "X" else "X"

--- ai.py ---
import random


class AI:
    def choose_column(self, board, me="O", opponent="X"):
        # Starter AI is deliberately basic; students improve it in
Task 3.
        legal = [c for c in range(7) if board.grid[0][c] == "."]
        return random.choice(legal) if legal else None

--- main.py ---
from game import Game

if __name__ == "__main__":
    Game().run()

Do NOT change anything yet. Explain how a move travels
from the user's input in
game.py to board.drop() and then to winner(). Then list any
bugs or missing
behaviour you can see.
…
```

Your game is structured well into four files, with each file handling a specific responsibility. I'll explain the flow without changing any code, then list the bugs and missing behaviours I can identify.

1. How a move travels through the game

`main.py`
Creates the Game object and calls `Game().run()`.

↓

`game.py` — `Game.run()`
Displays the board and asks the human for a column when it's X's turn.

↓

Input validation and conversion
Converts the user's input from a string to a zero-based column index.

↓

`board.py` — `Board.drop(col, token)`
Checks the column and places the token in the lowest available cell.

↓

Back to `game.py`
Calls `winner()` to check for a win, then `full()` to check for a draw. If neither occurs, switches turns.

Step 1: The game starts in `main.py`

Python

```python
from game import Game

if __name__ == "__main__":
    Game().run()
```

- `Game()` creates a new game.
- `run()` starts the main game loop.

Inside `Game.__init__()`, three things are initialized:

```python
self.board = Board()
self.ai = AI()
self.turn = "X"
```

This creates an empty board, an AI player, and sets the human player (`X`) to go first.

Step 2: The user enters a column in `game.py`

The board is printed, and because `self.turn == "X"`, this branch runs:

```python
raw = input("Column (1-7), or q: ").strip().lower()
```

Suppose the user enters `4`.

- `input()` reads the entry as a string.
- `.strip()` removes leading and trailing whitespace.
- `.lower()` converts the text to lowercase, allowing `q` to be recognized regardless of case.

If the user enters `q`, the game exits.

Otherwise, the input is converted to a zero-based index:

```python
col = int(raw) - 1
```

For example:

User input	Value of `col`
`1`	`0`
`4`	`3`
`7`	`6`

This conversion is necessary because users count columns from 1, but Python lists use indexes starting at 0.

If the input cannot be converted to an integer, `ValueError` is caught and the user is prompted again.

Step 3: `game.py` calls `board.drop()`

After the column is obtained, the game executes:

```python
if col is None or self.board.drop(col, self.turn) is None:
    print("Column unavailable.")
    if self.turn == "O":
        return
    continue
```

For our example, `col = 3` and `self.turn = "X"`.

The method call is effectively:

```python
self.board.drop(3, "X")
```

Inside `board.py`, the method first checks whether the column is within the valid range:

```python
```

```python
if not 0 <= col < COLS:
    return None
```

Since `COLS = 7`, valid indexes are `0` through `6`.

Next, it searches from the bottom row upward:

Python

```python
for r in range(ROWS - 1, -1, -1):
    if self.grid[r][col] == ".":
        self.grid[r][col] = token
        return r
```

Since `ROWS = 6`, the search starts at row `5` and moves toward row `0`.

If column 4 is empty, the first token is placed at `self.grid[5][3]`.

Example: after X drops into column 4

```
1 2 3 4 5 6 7
. . . . . . .
. . . . . . .
. . . . . . .
. . . . . . .
. . . . . . .
. . . X . . .
```

The token falls to the bottom because the board searches upward from row 5.

The method returns the row index where the token was placed, which is `5` in this example.

Important: `drop()` does not itself check for a winner. It only places the token and returns the row index, or `None` if the move cannot be made.

Step 4: `game.py` **calls** `winner()`

After a successful move, execution continues to:

```python
if self.board.winner(self.turn):
    self.board.print()
    print(self.turn, "wins!")
    return
```

For the human player's move, the call is:

```python
self.board.winner("X")
```

The method checks every cell containing `X` and tests the directions listed here:

```python
directions = [(0, 1), (1, 0)]
```

These represent:

- `(0, 1)` — horizontal, moving right.
- `(1, 0)` — vertical, moving down.

For each matching cell, the method checks four consecutive positions, including the starting cell.

If all four contain the requested token, it returns `True`. Otherwise, after checking the board, it returns `False`.

One important limitation: the code checks only horizontal and vertical wins. Connect Four also allows diagonal wins, which are currently missing.

Step 5: The game checks for a draw and changes turns

If nobody has won, the game executes:

```python
if self.board.full():
    self.board.print()
    print("Draw.")
    return
```

`full()` checks whether every cell in the top row is occupied. If so, no column can accept another token.

If the board is not full, the turn switches:

```python
self.turn = "O" if self.turn == "X" else "X"
```

The loop repeats, displaying the updated board and asking for the next move.

On `O`'s turn, `game.py` calls `self.ai.choose_column(self.board)`. The AI chooses a column, and that column follows the same `drop()` → `winner()` → `full()` sequence.

---

2. Bugs and missing behaviour

Here are the issues I can identify from the code as written, without making any changes.

⚠ **1. Diagonal wins are not detected**

Functional bug

The `winner()` method checks horizontal and vertical directions only. It misses both diagonal directions, so a valid four-in-a-row diagonal will not be recognized.

⚠ **2. Invalid numeric input is not handled gracefully**

Input handling

If the user enters `0`, `8`, or a negative number, the conversion succeeds, but the column index is invalid. `drop()` rejects it, and the game prints `Column unavailable.`

This does not crash, but the message is misleading: the number is outside the allowed range rather than identifying a full column.

---

3. The AI may terminate the game unexpectedly

Robustness

If `choose_column()` returns `None`, or returns a column that cannot accept a token, the following code exits the game:

```python
if self.turn == "O":
    return
```

The current random AI normally avoids full columns and returns `None` only when no legal columns remain. However, this branch makes the game silently end if the AI ever returns an invalid move.

---

4. The AI plays randomly

Missing feature

`ai.py` chooses a random legal column. It does not check whether it can win immediately, block the human's winning move, or prioritize strategically useful columns.

It is a valid starter AI, but not a competitive opponent.

---

5. No restart or replay option

Once someone wins or the board is full, `run()` returns and the game ends. The user must run the program again to play another round.

6. No score tracking or player statistics

The game announces the winner or a draw, but it does not keep track of wins, losses, or draws across rounds.

7. No handling for end-of-input

If the input stream closes—for example, the user presses Ctrl+D on Linux/macOS or Ctrl+Z followed by Enter on Windows—`input()` can raise `EOFError`. The game does not catch it.

3. A few additional observations

- **The board dimensions are hard-coded in the AI.** `board.py` defines `COLS = 7`, but `ai.py` uses `range(7)`. It works for the current board, but the AI would need updating if the board dimensions changed.
- `winner()` **is more limited than it first appears.** It only checks runs starting from a matching token and moving right or down. That is sufficient for horizontal and vertical wins, because every four-token run has a leftmost or topmost starting point. The issue is the missing diagonal directions, not the right/down approach itself.
- **The game checks wins in the correct order.** It checks for a win before checking whether the board is full, so a winning move that fills the last available cell is correctly reported as a win rather than a draw.
- **The board uses a simple and readable representation.** `"."` represents an empty cell, `"X"` represents the human, and `"O"` represents the AI. `drop()` and `full()` are straightforward and fit the current implementation.

4. What I would prioritize

If you're planning to improve this game as a programming task, I'd tackle the issues in this order:

Priority	Task	Reason
1	Add both diagonal win checks	Essential Connect Four functionality
2	Improve input validation and error messages	Better user experience
3	Improve AI move selection	Makes the game more challenging
4	Handle unexpected AI moves and end-of-input	Improves robustness
5	Add replay and score tracking	Useful additional features

Bottom line: your current program has a sound basic flow, and the separation between `main.py`, `game.py`, `board.py`, and `ai.py` is sensible. The most significant functional bug is the missing diagonal win detection. The AI's random play is an intentional limitation, while the remaining points are primarily robustness and usability improvements.

Fix the bug in Board.winner() in board.py. It currently only detects horizontal
and vertical wins. Make it detect all four directions of four-in-a-row, including
both diagonals (down-right and down-left), without breaking horizontal and
vertical detection. Shorter sequences (3 in a row) must not count as a win.
Keep the change minimal, do not rewrite other files, and show me only the updated
winner() method. Then give me 5 test positions (horizontal, vertical, both
diagonals, and a 3-in-a-row that should NOT win) I can use to verify it.

Updated `winner()` method in `board.py`

Python

```python
def winner(self, token):
    directions = [(0, 1), (1, 0), (1, 1), (1, -1)]

    for r in range(ROWS):
        for c in range(COLS):
            if self.grid[r][c] != token:
                continue

            for dr, dc in directions:
                if all(
                    0 <= r + i * dr < ROWS and
                    0 <= c + i * dc < COLS and
                    self.grid[r + i * dr][c + i * dc] == token
                    for i in range(4)
                ):
                    return True
```

5 test positions

Use `X` as the token and call `board.winner("X")`. Each test uses a fresh board. Rows are numbered **1–6 from top to bottom**, and columns **1–7 from left to right**.

Test	Position of `X`	Expected
1. Horizontal	Row 6, columns 1–4	`True`
2. Vertical	Column 1, rows 3–6	`True`
3. Diagonal down-right	(row 1, col 1), (row 2, col 2), (row 3, col 3), (row 4, col 4)	`True`
4. Diagonal down-left	(row 1, col 4), (row 2, col 3), (row 3, col 2), (row 4, col 1)	`True`
5. Three in a row	Row 6, columns 1–3	`False`

Why it works: The four direction vectors cover horizontal, vertical, down-right, and down-left. The method checks exactly four consecutive cells, so three matching cells alone cannot trigger a win.

Now handle game termination consistently in game.py (and board.py if needed).
Requirements:
- Handle draws (board full, no winner), full columns, out-of-range columns, and
  non-numeric input.
- Invalid input must never change the board or switch the turn; the player is
  simply asked again with a clear message.
- A winning move must end the game before another turn is requested.
- Quitting with 'q' (and Ctrl+C / Ctrl+D) should exit cleanly with a message.
- Every terminal state (win, draw, quit) should print a clear message.
Keep the existing structure and keep all state in memory.
Show me the full
updated game.py and any changed methods in board.py.

The cleanest fix is to keep `Board` responsible for validating and placing moves, while `Game.run()` handles input, turn switching, wins, draws, and quitting.

The key rule is that **an invalid move must not modify the board or change the current player's turn**. A successful move is checked for a win immediately, before the game can request another move.

I'll preserve your existing four-file structure and keep everything in memory.

1. Updated `game.py`

Replace the contents of `game.py` with the following:

Python

```
from board import Board
from ai import AI


class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four — you are X.")

        try:
            while True:
                self.board.print()
```

2. Changes needed in `board.py`

No additional changes are required to `board.py`, provided you have the existing methods from your original code and the updated `winner()` method from the previous step.

Your existing `drop()` already handles invalid column indexes and full columns by returning `None`, without changing the board.

The updated `winner()` should retain all four directions:

Python

```
directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
```

This preserves horizontal and vertical detection and adds both diagonals.

3. Expected behaviour

Situation	Behaviour
Non-numeric input, e.g. `abc`	Shows an error; asks again
Out-of-range input, e.g. `0` or `8`	Shows a range error; asks again
Full column	Shows an unavailable-column message; asks again
Invalid move	Board and turn remain unchanged
Four consecutive tokens	Prints the board and winner; exits
Full board with no winner	Prints the board and draw message; exits
Enter `q`	Prints a quit message; exits
Ctrl+C	Prints an interruption message; exits
Ctrl+D / end-of-input	Prints an input-closed message; exits

One implementation detail worth noting: the AI's invalid-move branch retries without switching turns. Your current random AI selects only legal columns, so that branch is primarily a safeguard.

To run the game:

```bash
python main.py
```

No external libraries, files, or persistent storage are needed. All game state remains in memory.

Improve ai.py so the AI makes meaningful decisions, in this priority order:
1. If the AI ("O") can win immediately, play that column.
2. Otherwise, if the player ("X") would win next move, block that column.
3. Otherwise, prefer columns closest to the centre.
The AI must never choose a full or out-of-range column, and should return None
only if the board is full. Do not add external libraries and do not print

anything from inside the AI. If you need helper methods on Board (like listing
legal columns or undoing a trial move), add them to board.py as small methods.

We'll improve the AI using a simple **win** → **block** → **centre** strategy, without minimax or external libraries.

The AI will:

1. Try each legal column and choose an immediate winning move for `O`.
2. If there is no immediate win, simulate `X` playing each legal column and block an immediate win.
3. Otherwise, choose the legal column closest to the centre, preferring column 4 when equally good options exist.
4. Return `None` only when the board is full.

To keep the existing structure, we can implement trial moves by temporarily placing a token and restoring the cell afterward. No extra `Board` methods are needed.

Updated `ai.py`

Replace your current `ai.py` with this:

Python

```python
class AI:
    def choose_column(self, board, me="O", opponent="X"):
        # Find all legal columns, from left to right.
        legal = [
            c for c in range(len(board.grid[0]))
            if board.grid[0][c] == "."
        ]

        # No legal moves means the board is full.
        if not legal:
            return None

        # Priority 1: Play a winning move if one exists.
        for col in legal:
            row = board.drop(col, me)
```

Why this works

- **Winning takes priority:** If `O` can win immediately, the AI returns that column without considering blocks.
- **Blocking comes second:** If `O` cannot win immediately, the AI finds a column where `X` would win on its next move and blocks it.
- **Centre comes third:** If neither situation applies, the AI chooses the legal column closest to the centre.
- **Trial moves are undone:** Each simulated move is removed before the AI returns or tests another column. The actual board remains unchanged while the AI decides.
- **No invalid moves:** Only columns whose top cells are empty are considered, so the AI cannot select a full or out-of-range column.
- **No unnecessary output:** The AI never prints anything.

No changes to `board.py` are needed. Your existing `drop()` and updated four-direction `winner()` methods provide everything the AI needs.

One limitation: this AI considers immediate wins and blocks only. It does not yet anticipate multi-move traps or avoid moves that allow the opponent to win on the following turn.

Add concise feedback for each successful disc placement, e.g.
"X placed a disc in column 3." Requirements: it prints exactly once per actual
move (both the player's and the AI's), and it must NOT print during win
detection or the AI's trial moves. Put the message in game.py after a
successful drop, not inside Board.drop(). Show only the changed lines.

Add this line in `game.py`, immediately after the successful `drop()` check (after the `if row is None:` block):

Python

```python
print(f"{self.turn} placed a disc in column {col + 1}.")
```

This prints exactly once per successful actual move, for both `X` and `O`. AI trial moves remain silent because they happen inside `ai.py`.

give me the final codes for each of the .py files

Worked for 8s

Here are the final versions of all four files for your terminal Connect Four game. They include:

- Horizontal, vertical, and both diagonal win detection.
- Input validation, full-column handling, draws, and clean quitting.
- An AI that prioritizes winning, blocking, and then choosing the centre.
- Exactly one placement message per successful actual move, with no output from AI trial moves.
- No external libraries or persistent storage.

Keep all four files in the same folder.

1. `board.py`

Python

```python
def __init__(self):
    self.grid = [["."] * COLS for _ in range(ROWS)]

def drop(self, col, token):
    if not 0 <= col < COLS:
        return None

    for r in range(ROWS - 1, -1, -1):
        if self.grid[r][col] == ".":
            self.grid[r][col] = token
            return r
```

2. `ai.py`

Python

```python
class AI:
    def choose_column(self, board, me="O", opponent="X"):
        # Find all legal columns.
        legal = [
            c for c in range(len(board.grid[0]))
            if board.grid[0][c] == "."
        ]

        # Return None only when the board is full.
        if not legal:
            return None

        # Priority 1: Win immediately if possible.
        for col in legal:
            row = board.drop(col, me)
```

3. `game.py`

Python

```python
from board import Board
from ai import AI


class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four — you are X.")

        try:
            while True:
                self.board.print()
```

4. `main.py`

Python

```
from game import Game

if __name__ == "__main__":
    Game().run()
```

Run the game

Make sure the four files are in the same directory, then open a terminal in that directory and run:

```bash
python main.py
```

On some systems, you may need:

```bash
python3 main.py
```

Note: This AI is intentionally simple. It wins when possible, blocks immediate threats, and otherwise prefers the centre. It does not search several moves ahead.

Write a simple test script using Python's built-in unittest module (no external
libraries) covering: horizontal, vertical, and both diagonal wins; a 3-in-a-row
that is not a win; a draw position; dropping into a full column; invalid input
handling; the AI taking an immediate win; the AI blocking an immediate player
win; and quitting. Tell me how to run it.

Worked for 15s

Analyzed

Analyzed

Analyzed

We'll create a single `test_game.py` using Python's built-in `unittest` module. It will test `Board`, `AI`, and `Game` without changing your existing game files.

For the game input tests, we'll use `unittest.mock` to simulate keyboard input, so you won't need to play manually. The tests will verify that invalid input doesn't change the board or switch turns, and that quitting exits cleanly.

1. Create `test_game.py`

Create a new file named `test_game.py` in the same folder as `main.py`, `game.py`, `board.py`, and `ai.py`.

Copy this code into it:

Python

```python
import unittest
from unittest.mock import patch
from contextlib import redirect_stdout
from io import StringIO

from board import Board
from ai import AI
from game import Game


class TestBoard(unittest.TestCase):

    def setUp(self):
        self.board = Board()

    def test_horizontal_win(self):
```

2. How to run the tests

Open your terminal in the folder containing all five Python files.

Run:

Bash

```
python -m unittest -v test_game.py
```

If your system uses `python3`, run:

```
Bash

python3 -m unittest -v test_game.py
```

The `-v` flag displays each test name and its result.

3. What the tests cover

Test category	What it verifies
Horizontal win	Four horizontal discs are detected
Vertical win	Four vertical discs are detected
Down-right diagonal	Diagonal win is detected
Down-left diagonal	Opposite diagonal win is detected
Three in a row	Three discs do not count as a win
Draw	Full board without a winner is recognized
Full column	Failed placement returns `None` without changing the board
Invalid column	Negative and out-of-range columns are rejected
Invalid user input	Non-numeric and out-of-range inputs preserve the board and turn
AI immediate win	AI chooses its winning move
AI blocking	AI blocks an immediate player win
AI full-board behaviour	Returns `None` when the board is full
Quit and EOF	`q` and Ctrl+D-style end-of-input exit cleanly
Keyboard interrupt	Ctrl+C exits cleanly

The script contains **15 test cases**. A passing test appears as `ok`; failures show an error or assertion explaining what went wrong.

One note: the draw test checks a full-board arrangement directly, rather than simulating every move that could produce it. This is intentional because the test targets the board's

draw-detection logic.

